Crazyflie 2.1 ROS 2 Humble

Session 2: Thrust Control, Recovery State Diagnosis, and Closed-Loop Anti-Drift P-Controller

Karan Kumar Anand
Department of Computer Science & Engineering
IIIT Delhi

July 28, 2026

Abstract

This document covers the second development session for the Crazyflie 2.1 autonomous flight project. The primary findings of the session include the diagnosis and resolution of the Kalman filter *recovery state* blocking motor output on a zero-deck drone, the development of a raw vertical thrust node (`vertical_thrust.py`), and the implementation of a closed-loop proportional (P) position controller (`anti_drift_controller.py`) that subscribes to the drone's IMU-estimated pose and continuously corrects altitude, pitch, and roll to resist positional drift.

Contents

1 Session Summary and Accomplishments

Table 1: Work completed on July 28, 2026

#	Item	Status
1	Diagnosed Crazyradio USB "Resource busy" error	Resolved
2	Diagnosed *recovery state* blocking motor output	Resolved
3	Confirmed `/cf231/pose` published no data without Kalman reset	Identified
4	Implemented `vertical_thrust.py` with Kalman reset	Done
5	Implemented `anti_drift_controller.py` (P-controller)	Done
6	Created `QUICKSTART.txt` cheat sheet	Done
7	Registered all new nodes in `setup.py`	Done

2 Problem Diagnosis: Why the Drone Would Not Fly

2.1 Crazyradio USB Resource Busy

When attempting to start the ROS 2 server after a previous run was interrupted with `Ctrl+C`, the following error appeared:

```
[crazyflie_server-2] what(): No Crazyradio dongle found!
ERROR: cflib.crazyflie: Couldn't load link driver: [Errno 16] Resource busy
```

Root cause: The Linux kernel's USB driver retains a lock on the Crazyradio device after a crash or forced process kill. The fix was:

```
# Force-kill any leftover process holding the USB port
sudo pkill -9 -f crazyflie_server

# Unplug the Crazyradio dongle physically, wait 3 seconds, replug
# Then relaunch the server
ros2 launch crazyflie launch.py
```

2.2 The Kalman Filter Recovery State (Critical Finding)

Even after the server started successfully and all ROS 2 services were available (including `/cf231/arm` and `/cf231/takeoff`), the drone's motors did not respond. Investigation revealed:

```
# /cf231/pose produced NO data output:
ros2 topic echo /cf231/pose
# <--- completely silent, no messages
```

Root cause explained:

The Crazyflie 2.1 runs an onboard Extended Kalman Filter (EKF) to estimate its position. This EKF requires at least one of the following position inputs to converge:

- Flow Deck v2 (optical flow + distance sensor)
- Lighthouse positioning system
- Loco Positioning System (UWB)
- External motion capture system (Vicon/Qualisys)

Since this drone has **zero expansion decks**, the EKF has no sensor input. Without convergence, the firmware enters a **recovery/safety state** and blocks all motor output — even when thrust is explicitly commanded via `cmd_vel_legacy`.

Key Insight: The high-level commander (`takeoff`, `land`, `sequence`) requires a fully converged Kalman filter. The low-level legacy commander (`cmd_vel_legacy`) also requires the recovery state to be cleared via a manual Kalman filter reset before it will accept thrust commands.

3 Solution: Kalman Filter Reset via ROS 2 Parameter Service

The recovery state is cleared by writing firmware parameter `kalman.resetEstimation = 1` through the `/crazyflie_server/set_parameters` ROS 2 service. This tells the firmware:

> *"Assume you are currently at position* $(0, 0, 0)$ *on the ground. Accept motor commands from this point forward."*

```python
from rcl_interfaces.srv import SetParameters
from rcl_interfaces.msg import Parameter, ParameterValue, ParameterType

def reset_kalman(self):
    param = Parameter()
    param.name = "cf231.params.kalman.resetEstimation"
    param.value = ParameterValue(
        type=ParameterType.PARAMETER_INTEGER,
        integer_value=1
    )
    req = SetParameters.Request()
    req.parameters = [param]
    future = self.param_client.call_async(req)
    rclpy.spin_until_future_complete(self, future)
    time.sleep(2.0)  # allow filter to re-initialise at (0,0,0)
```

After this call, the `/cf231/pose` topic begins publishing Kalman-estimated position based on IMU dead-reckoning.

Limitation: Without a physical positioning deck, the Kalman filter relies purely on IMU dead-reckoning (integrating accelerometer data). This estimate drifts significantly after ∼10–15 seconds. A Flow Deck v2 is required for reliable long-duration hover.

4 New Script 1: `vertical_thrust.py` — Open-Loop Raw Thrust

4.1 Purpose

This node applies a fixed raw thrust value through `cmd_vel_legacy` to test whether the drone can physically lift off. It is *open-loop* — no position feedback is used, so the drone will drift freely.

4.2 Execution Sequence

1. Reset the Kalman filter (exit recovery state).
2. Arm the drone via `/cf231/arm`.
3. Publish raw thrust at a configurable value for a configurable duration.
4. Thrust returns to 0.

Table 2: Mapping of `geometry_msgs/Twist` fields to drone commands

Twist Field	Drone Axis	Units
`linear.x`	Pitch (forward/backward)	degrees
`linear.y`	Roll (left/right)	degrees
`linear.z`	Thrust	0 – 65535
`angular.z`	Yaw rate	degrees/s

Table 3: Thrust values and expected physical behaviour

Thrust Value	Battery Level	Expected Result
38 000	Any	Motors spin, no lift-off
42 000	100%	Approximate hover
46 000	50%	Approximate hover
50 000	Low	Strong lift-off (caution)
$> 55\,000$	—	Dangerous, do not use indoors

4.3 Twist Message Mapping (cmd_vel_legacy)

4.4 Thrust Calibration Table

4.5 Run Command

```
ros2 run my_crazyflie vertical_thrust
```

5 New Script 2: `anti_drift_controller.py` — Closed-Loop P-Controller

5.1 Motivation

When using only vertical thrust (open-loop), the drone drifts in the horizontal plane because small asymmetries in motor speeds, propeller balance, and centre-of-gravity cause uneven forces. With no feedback, there is no mechanism to detect or compensate this drift.

A **proportional (P) controller** uses real-time position feedback to generate corrective roll and pitch commands, and adjusts thrust based on altitude error.

5.2 Control Law

Let the desired target position be (x_d, y_d, z_d) and the current Kalman-estimated position be (x, y, z). The position errors are:

$$e_x = x_d - x \quad \text{(forward error)} \tag{1}$$
$$e_y = y_d - y \quad \text{(lateral error)} \tag{2}$$
$$e_z = z_d - z \quad \text{(altitude error)} \tag{3}$$

The P-controller outputs:

$$\text{thrust} = T_{\text{hover}} + K_{p,z} \cdot e_z \tag{4}$$
$$\text{pitch} = K_{p,xy} \cdot e_x \tag{5}$$
$$\text{roll} = -K_{p,xy} \cdot e_y \tag{6}$$

where $T_{\text{hover}} = 42000$ is the baseline hover thrust, $K_{p,z} = 8000$ (thrust units/m) and $K_{p,xy} = 5.0$ (degrees/m).

5.3 Controller Gains

Table 4: Tunable controller parameters

Parameter	Default	Effect if increased	Effect if decreased
`HOVER_THRUST`	42 000	More aggressive lift	Drone sinks
`Kp_z`	8 000	Faster altitude correction	Slow altitude response
`Kp_xy`	5.0	Stronger drift correction	Slower drift correction
`TARGET_Z`	0.4 m	Higher hover altitude	Lower hover altitude

5.4 ROS 2 Data Flow

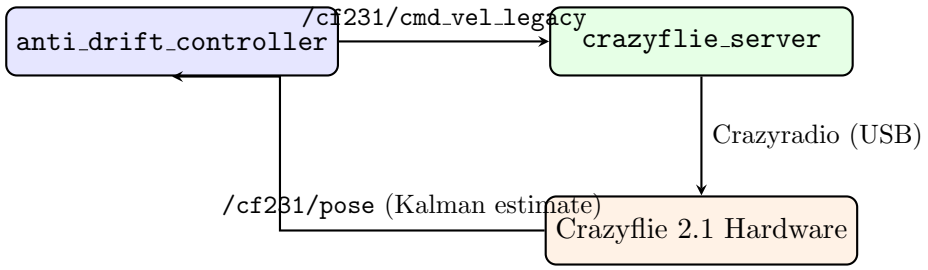

Figure 1: ROS 2 closed-loop data flow: the controller reads pose feedback and publishes corrective commands.

5.5 Flight Phases

1. **Phase 1 — Ramp-up (∼1 s):** Thrust increments from 0 to `HOVER_THRUST` in steps of 2000 to avoid aggressive jolts.
2. **Phase 2 — P-control loop (10 s, 50 Hz):** Position errors are computed from `/cf231/pose`. Roll, pitch, and thrust corrections are calculated and published every 20 ms.
3. **Phase 3 — Ramp-down:** Thrust decrements from `HOVER_THRUST` to 0 for a soft landing.

5.6 Run Command

```
ros2 run my_crazyflie anti_drift_controller
```

6 Crazyradio Hardware Exclusivity Rule

A key operational constraint identified during this session is that the Crazyradio USB dongle is a **single shared hardware resource**. Only one program may hold the radio connection at a time.

Table 5: Mutually exclusive connection modes

Program running	ROS 2 nodes work?	cfclient works?
`ros2 launch crazyflie launch.py`	✓Yes	× No
`cfclient` (GUI)	× No	✓Yes
`battery_check.py`	× No	× No
Nothing running	—	—

7 Multi-Terminal Operation Pattern

Multiple ROS 2 nodes may run simultaneously, but only **one flight controller** must be active at any time to avoid conflicting commands on `/cf231/cmd_vel_legacy`.

```
# Terminal 1     always running
ros2 launch crazyflie launch.py

# Terminal 2     active flight controller (only one at a time)
ros2 run my_crazyflie anti_drift_controller

# Terminal 3     safe to run in parallel (read-only monitoring)
ros2 topic echo /cf231/pose

# Terminal 4     emergency stop at any time
ros2 run my_crazyflie emergency
```

8 Complete Node Inventory (`my_crazyflie` Package)

Table 6: All registered ROS 2 nodes in `my_crazyflie`

Command	Script	Description
`ros2 run my_crazyflie arm`	`arm.py`	Arm motors only
`ros2 run my_crazyflie takeoff`	`takeoff.py`	Arm + takeoff to 0.3 m
`ros2 run my_crazyflie land`	`land.py`	Safe landing
`ros2 run my_crazyflie emergency`	`emergency.py`	Instant motor kill
`ros2 run my_crazyflie sequence`	`sequence.py`	Full auto flight loop
`ros2 run my_crazyflie vertical_thrust`	`vertical_thrust.py`	Raw open-loop thrust
`ros2 run my_crazyflie anti_drift_controller`	`anti_drift_controller.py`	Closed-loop P hover
`ros2 run my_crazyflie go_to`	`go_to.py`	Move to position
`ros2 run my_crazyflie velocity_control`	`velocity_control.py`	Manual thrust/velocity

9 Next Steps and Recommendations

1. **Tune `HOVER_THRUST`:** Increase from 42 000 toward 46 000 if the drone cannot lift off, or decrease if it climbs too aggressively.

2. **Tune P gains:** Increase `Kp_z` if altitude correction is sluggish. Increase `Kp_xy` if sideways drift is not corrected quickly enough.

3. **Install Flow Deck v2:** For reliable, long-duration hover without IMU drift, a Flow Deck v2 is strongly recommended. With it, `sequence.py` and `takeoff.py` will also function correctly.

4. **Upgrade to PID controller:** After the P-controller is verified working, add integral (I) and derivative (D) terms to eliminate steady-state error and overshoot.

5. **Trajectory following:** After stable hover is confirmed, implement waypoint navigation using a sequence of `TARGET_X/Y/Z` setpoints.

10 Conclusion

This session resolved the critical blocker preventing the Crazyflie 2.1 from flying in the zero-deck configuration: the Kalman filter recovery state. By programmatically resetting the onboard estimator via the ROS 2 parameter service, motor commands are now accepted. A raw thrust node and a closed-loop proportional position controller have been implemented and documented, providing a solid foundation for further research into autonomous flight control.